

3IMACS

UF I3MAAU12 Control of Linear Systems

Matlab Project

17 mars 2023

1 Objectives

The objective of this consultancy is twofold.

On the one hand, the goal is to introduce the use of the software *Matlab*, in the framework of automatic control. In addition to the basic functionalities of the software, you will be brought to complete your training by using the specific toolboxes that are *Control toolbox* and *Simulink*.

On the other hand, a second objective (perhaps more ambitious) consists in making a small synthesis of your knowledge in automatic control. Indeed, starting from a concrete case study of a two-wheeled robot built with LEGO Mindstorms, you will try to conduct a reflection that will require you, using different techniques of automatic control, to have a more global look on what you are doing. Thus, a particular attention will be asked to interpret the results as well as the adequacy of the tools used.

Throughout this study, you will have to write Matlab live scripts that will allow you to execute simulations as well as the plotting of results curves. The advantage of a live script is that it contains your code, output, and formatted text together in a single interactive environment called the Live Editor. In live scripts, you can write your code and view the generated output and graphics along with the code that produced it. You can also add formatted text, images, hyperlinks, and equations to create an interactive narrative that you can share with others.

All your work must be included in a Matlab live script that would be used as your report. An exported pdf version of the live script can also be provided as your report. Pay attention to include all your code, plots, analyses, comments, and conclusion in your file.

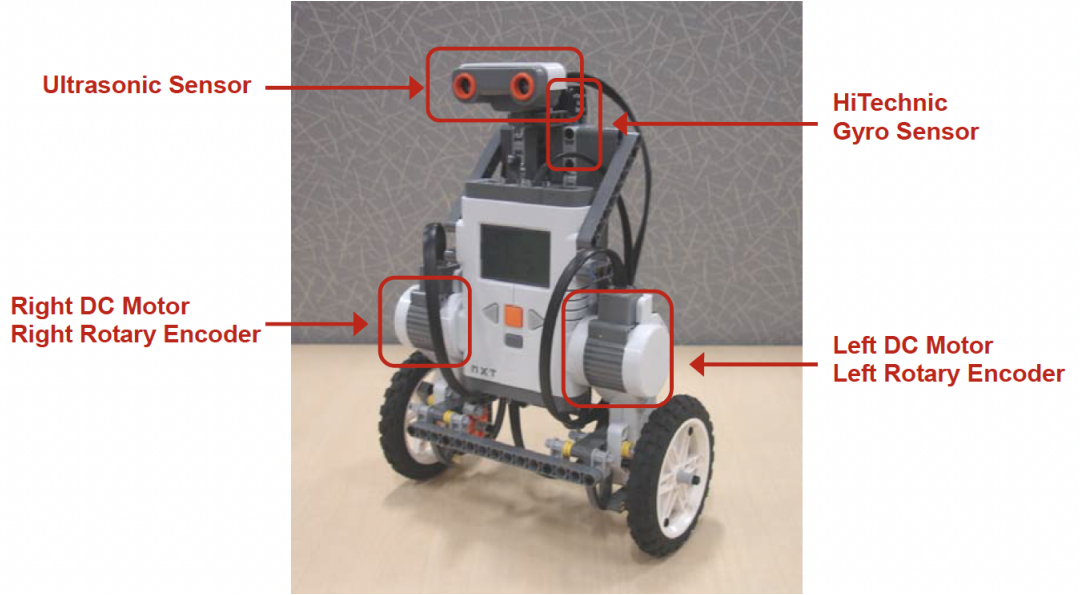


FIGURE 1 – The NXTway-GS system

2 The two-wheeled robot

The system considered in this project (see Figure 1) is a NXTway-GS self-balancing two-wheeled robot built with LEGO Mindstorms NXT.

The robot is equipped of several actuators and sensors :

- 2 DC Motors (right and left) whose speed is controlled by *PWM* (Pulse Width Modulation). The input of the DC motors is noted u .
- 2 rotary encoders (one for each wheel) that provide an angle measurement in degree. This measurement called the *DC_motor_angle* is denoted $\theta_{m_{l,r}}$ for the right and left motor respectively.
- An ultrasonic sensor that gives a distance measurement in centimeter to compute for instance the distance to a wall.
- A Gyro sensor that provides an angular velocity in degree per second. This sensor provides the body pitch angular velocity noted ψ .

The NXTway-GS can be considered as a two wheeled inverted pendulum model, shown Figure 2.

The side and plane view of the system are given Figure 3 with θ the average angle of left and tight wheel, Ψ the body pitch angle and ϕ the body yaw angle.

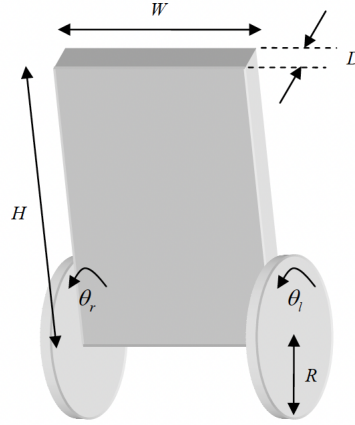


FIGURE 2 – Schematic view of the system as an inverted pendulum

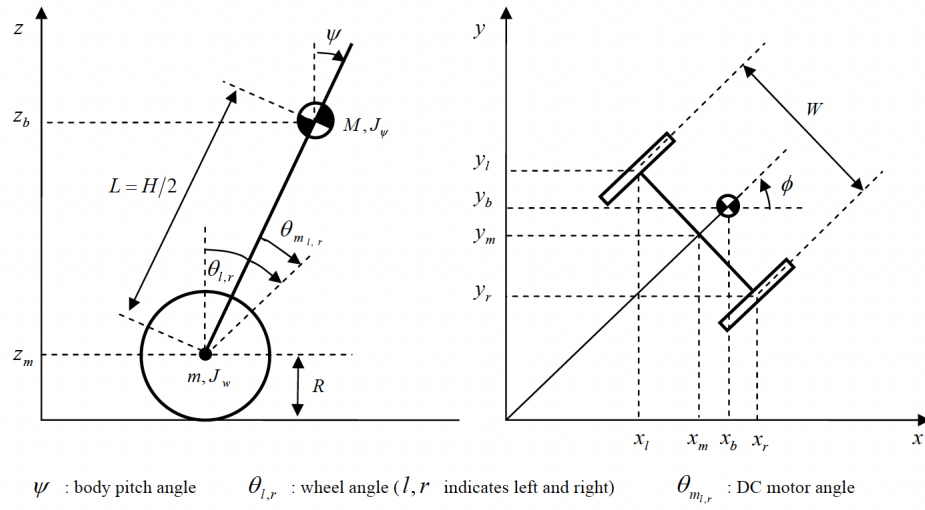


FIGURE 3 – Side and plane view of the two-wheeled inverted pendulum

3 Process modelling

We can derive motion equations of the two-wheeled inverted pendulum by the Lagrangian method based on the coordinate system given Figure 3. More details on this modelling step are given in the file Annexe available on Moodle. After a linearization step, the motion equations are approximated as follows :

$$[(2m + M)R^2 + 2J_w + 2n^2J_m]\ddot{\theta} + (MLR - 2n^2J_m)\ddot{\Psi} = F_\theta \quad (1)$$

$$(MLR - 2n^2J_m)\ddot{\theta} + (ML^2 + J_\Psi + 2n^2J_m)\ddot{\Psi} - MgL\Psi = F_\Psi \quad (2)$$

$$[\frac{1}{2}mW^2 + J_\phi + \frac{W^2}{2R^2}(J_w + n^2J_m)]\ddot{\phi} = F_\phi \quad (3)$$

where :

- m, M denote the wheel and the body weight (in Kg) respectively.
- W is the body wide (in meters).
- L the distance of the center of mass from the wheel axle in meters.
- R is the wheel radius (in meters).
- n is the gear ratio
- J_w, J_Ψ, J_ϕ and J_m are the inertia moment of the wheel, the body pitch, body yaw and DC motor respectively (expressed in $Kg.m^2$).
- θ the average angle of left and right wheel in radian.
- Ψ the body pitch angle in radian.
- ϕ the body yaw angle in radian.
- F_θ, F_Ψ and F_ϕ are the generalized forces.

These forces can be expressed using the motors' voltage (v_l and v_r) taking into account the friction coefficients f_m (between the body and DC motor) and f_w (between the wheel and the floor), but also the DC motor torque constant (noted K_t (N.m/A)) and the parameters of the DC motors (K_b the back EMF constant (V-s/rad) and R_m the motor resistance (ohms)).

$$F_\theta = \alpha(v_l + v_r) - 2(\beta + f_w)\dot{\theta} + 2\beta\dot{\Psi} \quad (4)$$

$$F_\Psi = -\alpha(v_l + v_r) + 2\beta\dot{\theta} - 2\beta\dot{\Psi} \quad (5)$$

$$F_\phi = \frac{W}{2R}\alpha(v_r - v_l) - \frac{W^2}{2R^2}(\beta + f_w)\dot{\phi} \quad (6)$$

with $\alpha = \frac{nK_t}{R_m}$ and $\beta = \frac{nK_tK_b}{R_m} + f_m$.

Looking at the motion equations (1) to (3), it appears that the two first equations depend only on Ψ and ϕ whereas the last one depends only on θ . It is then possible to model the system by considering two sub-systems given by :

$$E \begin{bmatrix} \ddot{\theta} \\ \ddot{\Psi} \end{bmatrix} + F \begin{bmatrix} \dot{\theta} \\ \dot{\Psi} \end{bmatrix} + G \begin{bmatrix} \theta \\ \Psi \end{bmatrix} = H \begin{bmatrix} v_l \\ v_r \end{bmatrix} \quad (7)$$

$$\left[\frac{1}{2}mW^2 + J_\phi + \frac{W^2}{2R^2}(J_w + n^2 J_m) \right] \ddot{\phi} + \frac{W^2}{2R^2}(\beta + f_w) \dot{\phi} = \frac{W}{2R} \alpha (v_r - v_l) \quad (8)$$

3.1 MIMO (Multiple Input Multiple Output) state space models

A data file `NXP_robot_parameters.m` containing the values of all the parameters of the two wheel robot and the matrices of the system is available on the Moodle page of the Labwork.

1. From the subsystem of (8) and by considering the state vector $x_2 = \begin{bmatrix} \phi & \dot{\phi} \end{bmatrix}^T$ and the input vector $u = \begin{bmatrix} v_l & v_r \end{bmatrix}^T$, create a live script section that generates the state space model $\dot{x}_2 = A_2 x_2 + B_2 u$.
2. From the subsystem given by (7)
 - by considering (1) and (2) generate the matrices E, F, G and H .
 - by considering the state vector $x_1 = \begin{bmatrix} \theta & \Psi & \dot{\theta} & \dot{\Psi} \end{bmatrix}^T$ and the input model $u = \begin{bmatrix} v_l & v_r \end{bmatrix}^T$ generate the state space model $\dot{x}_1 = A_1 x_1 + B_1 u$.

Some useful Matlab commands :

- `A = [-2 -1; 1 -2], X = zeros(n)`
- `sys = ss(A,B,C,D)`

3.2 SISO (Single Input Single Output) state space model

The models obtained previously consider an input vector $u = \begin{bmatrix} v_l & v_r \end{bmatrix}^T$ to control the trajectory of the robot, but also the balance.

In the remainder we will consider first only the problem of balance control assuming a same input u for each wheel given by :

$$u = \frac{v_l + v_r}{2}$$

1. Generate the state space model for the balance control problem.

4 Simulation of the system with Simulink

1. In a first step, set up a simulink application that will allow you to realize the open loop simulation of the of the system.
 - Create a blank simulink model. (New>> Simulink Model >> Blank Model).

- In the library browser, find the "state space" block and drag it into your model.
 - Choose a step signal as input and a "to workspace" block as output, as well as a scope.
 - The simulation parameters (final time, integration step,...) as well as the different parameters of the system (A , B , C , D , step value...) will be variables of the workspace and fixed in the live script.
 - Create a new section in your matlab live script to launch the simulation of the model and which can also trace the results.
2. Plot the step response of your model. Is the system stable in open-loop?
 3. Do the same for the impulse response.

Some useful Matlab commands :

- `sim('Mymodel')` for simulating Mymodel.slx model
- `plot(ans.simout), plot(x,y1,x,y2,'--',x,y3,':')`
- `subplot, tiledlayout, grid, hold`
- `xlabel ('Myxlabel'), ylabel ('Myylabel'), title ('MyTitle'), ...`

5 Open loop analysis

1. In a new section of your script, study observability and controllability of this system.
2. Compute and display the system's modes in the Laplace plane. Characterize those modes (damping, natural frequency, etc). Is the system stable in open-loop? Is it coherent with the responses you get in previous section?
3. Display the step response with the matlab command to retrieve the result of the previous section.
4. Do the same for the impulse response.

Some useful Matlab commands :

- `obsv(sys), ctrb(sys)`
- `rank(M), eig(M)`
- `pzmap(sys)`
- `ss2tf(sys), tf(num, den), zpk(F)`
- `damp(sys)`
- `isstable(sys), esort(p)`
- `step(sys), stepplot(sys), stepinfo(sys)`
- `impulse(sys), stepinfo(sys) ...`

6 Balance control

After a modeling and analysis phases, we aim now at controlling the system. The objective is to stabilize the upright position of the NXTway. To this end, a

closed-loop control law has to be designed to calculate the appropriate command signal u that maintains the angular position Ψ close to 0. At first, the control law will be based only on the output measure y . Then, the full state measurement x will be considered.

6.1 Proportional controller

A first classical controller is the proportional control law :

$$u = K_p \varepsilon$$

with $\varepsilon = y_{\text{ref}} - y$, y_{ref} being the setpoint for the output, that is, the desired average wheels angle (for instance, $y_{\text{ref}} = \frac{\pi}{2} \text{ rad}$). K_p is a positive constant gain.

1. Draw the block diagram of the closed-loop system with the above controller and the linearized system established in the previous section.
2. For some values of K_p between 0.1 and 10, calculate the closed-loop transfer function and its poles. What assumption can be made regarding the stability of the closed-loop system ?
3. Write a script that plots the poles of the closed-loop system with respect to K_p in the complex plane (root locus). The instruction `logspace` should be used to generate several values of the gain over a couple of decades $K_p \in [10^{-1}, 10^3]$. Compare your root locus with the result of the instruction `rlocus`. Can the system be stabilized ?
4. Simulate the system with the proportional control on Simulink for some arbitrary values of K_p . What do you observe ? Is it consistent with your calculation ?

6.2 Proportional Integral controller

Let us now consider a slightly more fancy law : a proportional-integral (PI) controller. It is a transfer function that includes two tuning parameters. It can be written in the Laplace domain as

$$u = \left(K_p + K_i \frac{1}{p} \right) \varepsilon$$

where K_p and K_i are the constant gains to be set. Calculate the characteristic polynomial of closed-loop system and apply the Routh criterion to derive stability conditions w.r.t. the two above parameters. Can the system be stabilized ?

6.3 State feedback controller

In this part we consider a state feedback controller. The objective is that the controlled system behaves as a second system order with an overshoot of 5% and a settling time around 0.7 seconds for a step input reference of $\pi/5$.

1. Design a state feedback controller $u = -Kx + y_c$ following the method seen during the lectures. Check that the specifications are satisfied.
2. Test the designed control law with Simulink
3. Modify the control law u to obtain a null steady state error.
4. Test the new control law with Simulink
5. The motors accept an input value in $[-8,+8]$ volts. Plot the command signal to eventually detect a saturation problem.
6. Determine the limit value of the settling time so that the state feedback with pregain control law satisfies the saturation constraint.

7 Nonlinear simulations

In the previous section, a state feedback control law has been designed based on a linear model of the robot. You have been able to verify your calculations with Simulink simulations and your linear state space model. A better validation step consists in testing your control on the nonlinear model. To this end, a Simulink file `NXTwaySim.slx` is provided (note that a specific version for Matlab 2020a is available if necessary). Make sure that the “Model Settings” are as follows :

- Solver type : **Fixed-step**
- Solver : **ode4 (Runge-Kutta)**
- Fixed-step size (fundamental sample time) : **0.0001**

A MATLAB function `playAnimationrobot.m` is also provided¹. It is useful for visualizing the behavior of the two-wheeled robot resulting from a given control. Call this function once the Simulink has been run :

```
>> playAnimationrobot(positions,simtime)
```

Variables `positions` and `simtime` are generated by the nonlinear model block in Simulink. The animation function should not be modified.

1. Analyze the new Simulink file and describe all of its components.
2. Run some open-loop simulations with the two proposed input signals and try the animation function.
3. Modify the Simulink file to program your control law and test it on the nonlinear model. Set the reference signal y_{ref} as a step signal, from 0 to π rad at $t = 1$ s. Does the output θ converge to the reference while the balance control (ψ) is stable? Is the animation consistent with graphs in Simulink?
4. Define the reference signal so the robot travels, from its initial position 0, a distance of 0.5 m in 8 s.

1. You can type `help playAnimationrobot` in MATLAB to have... some help.

5. The reference is now set to 0. Modify your Simulink diagram to test the response of the closed loop system to a disturbance on ψ . The disturbance will be modeled as a pulse signal of amplitude 5 *deg*, a period of 5 *s* and a duty cycle of 10% which is added to the measure of ψ .

8 Conclusion

Give your conclusions on the problem treated during this study office but also on the progress of the study office (interest, difficulty, adequacy in relation to the lessons of courses and TD of this year and previous years, clarity of the subject,).